FRUBBER POS

Documento de Diseño Técnico Detallado

Versión: 1.0

Estado: Aprobado

Fecha: 28 de septiembre de 2026

Responsable del producto: Jhoan Rivera

Propósito del documento

Este documento transforma la arquitectura aprobada de Frubber POS en un diseño implementable para los cinco proyectos de la solución. Define componentes internos, contratos, casos de uso, objetos de dominio, límites transaccionales, secuencias, políticas de errores, seguridad, concurrencia y puntos de prueba.

El diseño implementa la Visión y alcance 1.1, la SRS 1.1, las Reglas de negocio 1.0, la Arquitectura técnica 1.0 y el Diseño de base de datos 1.0 aprobados. No agrega módulos comerciales ni sustituye el futuro diseño de interfaz.

1. Fuentes, alcance y objetivos

1.1 Documentos normativos

• Documento de Visión y Alcance, versión 1.1 aprobada.

• Especificación de Requisitos de Software, versión 1.1 aprobada.

• Documento de Reglas de Negocio, versión 1.0 aprobada.

• Documento de Arquitectura Técnica, versión 1.0 aprobada.

• Documento de Diseño de Base de Datos, versión 1.0 aprobada, incluida la migración 001_Initial.sql.

1.2 Alcance del diseño

El documento cubre la estructura interna de Domain, Application, Infrastructure, Desktop y Tests; los contratos entre capas; los casos de uso aprobados; la coordinación de transacciones SQLite; el inicio de la aplicación; autenticación y autorización; productos e inventario; caja; ventas; historial y anulaciones; configuración e impresión; respaldo; prueba y licencia; registro técnico y manejo de errores.

1.3 Exclusiones

No define la composición visual final, colores, tamaños, atajos ni distribución de controles; esos detalles corresponderán al Diseño de Interfaz y Experiencia de Usuario. Tampoco incorpora servidor, cloud, IA, categorías, precio de compra, clientes, impuestos separados, pagos combinados, devoluciones parciales, egresos, reportes avanzados, exportaciones, restauración desde la interfaz ni funciones no aprobadas.

1.4 Objetivos de calidad

• Mantener las reglas puras aisladas de WPF, SQLite y Windows.

• Hacer explícitas las transacciones y evitar ventas, anulaciones o cierres parciales.
• Mantener la interfaz receptiva mediante async/await.
• Permitir pruebas de casos de uso sin abrir ventanas ni usar hardware real.
• Favorecer un diseño sencillo, trazable y acorde con el presupuesto.

2. Decisiones de diseño aprobadas

2.1 Línea base de arquitectura

Se mantienen C# y .NET 10 LTS con net10.0-windows, WPF con MVVM, monolito modular, SQLite, Microsoft.Data.Sqlite, Dapper, SQL parametrizado, publicación autocontenida x64, Inno Setup, DPAPI LocalMachine, ECDSA P-256/SHA-256, PBKDF2-HMAC-SHA-256, Serilog y los cinco proyectos aprobados.

2.2 P-DD-001 — Organización por caso de uso

Application utilizará una clase por caso de uso. Cada clase tendrá una única operación ExecuteAsync, un Request inmutable y un Response específico. Las clases se agruparán por funcionalidad. No se incorporarán MediatR, bus de mensajes ni infraestructura CQRS adicional.

2.3 P-DD-002 — Composición de dependencias

Desktop utilizará Microsoft.Extensions.DependencyInjection. App será el único punto de composición: registrará casos de uso, puertos, implementaciones de Infrastructure, ViewModels, navegación y servicios de presentación. Ningún otro proyecto actuará como localizador de servicios.

2.4 P-DD-003 — Resultados y errores esperados

Los casos de uso devolverán Result<T>. Validaciones, autenticación, autorización, ausencia de datos, conflictos y bloqueos de licencia se representarán mediante códigos explícitos. Las excepciones se reservarán para fallos técnicos inesperados y se transformarán en un resultado seguro en el límite de Application o Desktop.

2.5 P-DD-004 — Instancia única

Solo podrá ejecutarse un proceso de Frubber POS por equipo. Un mutex nominal exclusivo se adquirirá antes de abrir la base de datos. Si ya está ocupado, la segunda ejecución mostrará un aviso y terminará sin modificar datos.

2.6 P-DD-005 — Ejecución asíncrona

Los contratos de entrada/salida devolverán Task y recibirán CancellationToken. Los ViewModels usarán comandos asíncronos y evitarán bloquear el hilo de interfaz. No se utilizarán async void salvo manejadores propios de WPF. Una operación ya confirmada en SQLite nunca se presentará como cancelada de manera ambigua.

2.7 P-DD-006 — Primer administrador

Cuando la tabla User esté vacía, el inicio mostrará un asistente único que solicitará nombre y PIN numérico de cuatro dígitos. El caso de uso creará exactamente un usuario activo con rol ADMIN.

Después de existir al menos un usuario, esa ruta quedará deshabilitada y el acceso normal exigirá autenticación.

2.8 P-DD-007 — Protección del último administrador

Se impedirá eliminar, desactivar o cambiar a CASHIER al último administrador activo. La comprobación se repetirá dentro de la misma transacción de escritura para evitar que la interfaz sea la única barrera.

3. Modelo de componentes

3.1 FrubberPOS.Domain

Contendrá entidades, objetos de valor, enumeraciones, validadores y servicios de dominio puros. No tendrá referencias a Application, Infrastructure, Desktop, Dapper, SQLite, WPF, Serilog ni API de Windows.

3.2 FrubberPOS.Application

Contendrá casos de uso, Request/Response, Result<T>, códigos de error, autorización, coordinación transaccional, DTO y puertos requeridos. Dependerá únicamente de Domain.

3.3 FrubberPOS.Infrastructure

Implementará los puertos de Application para SQLite/Dapper, migraciones, respaldo, PBKDF2, DPAPI, huella del equipo, ECDSA, reloj, impresión, archivos y registro. Dependerá de Application y Domain.

3.4 FrubberPOS.Desktop

Contendrá App, ventanas, vistas XAML, ViewModels, comandos, navegación, adaptadores de diálogo y el punto de composición. Dependerá de Application y referenciará Infrastructure solo para registrar implementaciones. No contendrá SQL ni cálculos comerciales críticos.

3.5 FrubberPOS.Tests

Contendrá pruebas unitarias de Domain y Application y pruebas de integración de Infrastructure. Podrá referenciar los proyectos necesarios, pero no formará parte de la distribución.

3.6 Reglas de dependencia

Domain no depende de otros proyectos. Application depende de Domain. Infrastructure depende de Application y Domain. Desktop depende de Application y utiliza Infrastructure únicamente desde el punto de composición. Se prohíben ciclos, Service Locator, acceso de ViewModels a Dapper y referencias de Domain a tipos de persistencia.

3.7 Diagrama UML de componentes

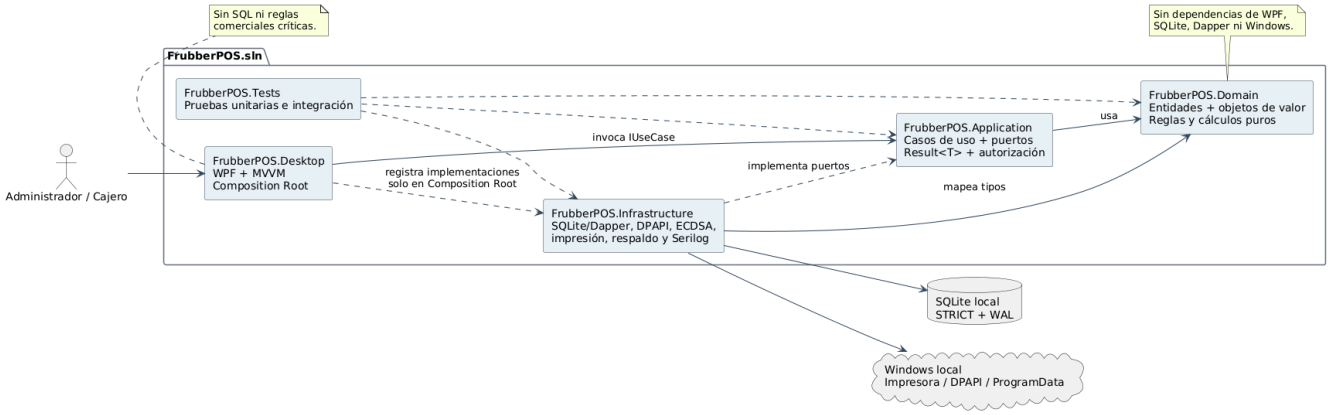


Figura 1. Componentes, dependencias y adaptadores de Frubber POS.

4. Organización del código

4.1 Espacios de nombres

Los espacios de nombres seguirán FrubberPOS.<Proyecto>.<Funcionalidad>. Los nombres del código estarán en inglés; los textos visibles y mensajes de validación estarán en español.

4.2 Organización por funcionalidad

Application se organizará en carpetas como Authentication, Users, Products, Inventory, Cash, Sales, History, Settings, Backup, Licensing y Startup. Cada carpeta contendrá sus casos de uso, contratos y DTO relacionados. No se creará un servicio global con responsabilidades de todo el sistema.

4.3 Convenciones de casos de uso

Cada clase terminará en UseCase; los modelos de entrada y salida terminarán en Request y Response. Un caso sin entrada utilizará un Request vacío explícito. Los Request serán records inmutables y no contendrán controles WPF, conexiones, transacciones ni entidades Dapper.

4.4 Inyección por constructor

Todas las dependencias obligatorias se recibirán por constructor. Se prohíben dependencias opcionales ocultas, estado global mutable y resolución directa desde IServiceProvider fuera de App y de fábricas expresamente justificadas.

5. Contratos transversales de Application

5.1 Contrato de caso de uso

La forma conceptual será IUseCase<TRequest,TResponse>.ExecuteAsync(TRequest request, CancellationToken cancellationToken), con retorno Task<Result<TResponse>>. No se exige que todos los casos implementen una interfaz genérica si ello perjudica la legibilidad, pero conservarán esa firma semántica.

5.2 Result<T>

Result<T> será éxito con un valor T o fallo con un Error; nunca ambos. Error contendrá ErrorCode, mensaje seguro para el usuario y, cuando aplique, el campo relacionado. No transportará PIN, sal, hash, clave de licencia completa, identificadores físicos completos ni SQL interno.

5.3 Códigos de error

• Validation: entrada corregible.

• Authentication: PIN o sesión no válidos.

• Authorization: rol sin permiso.

• NotFound: entidad inexistente.

• Conflict: estado concurrente o regla que ya no se cumple.

• LicenseBlocked: prueba, licencia o reloj bloquean la operación.

• Storage: fallo controlado de persistencia.

• Printing: venta guardada, pero impresión fallida.

• Backup: copia no creada o no validada.

• Unexpected: fallo no previsto con identificador de diagnóstico.

5.4 Excepciones

Domain podrá lanzar excepciones únicamente al intentar construir un estado imposible por una ruta interna; Application deberá prevenirlas mediante validación. Infrastructure podrá propagar excepciones técnicas tipadas. El límite exterior registrará el detalle técnico y devolverá un mensaje seguro. No se capturarán excepciones para ignorarlas.

5.5 Cancelación

Las consultas y operaciones previas al commit respetarán CancellationToken. Una cancelación durante una transacción provocará rollback. Después de que COMMIT haya finalizado, el caso devolverá éxito aunque la solicitud de cancelación llegue tarde; la impresión posterior podrá fallar o cancelarse sin alterar la venta.

5.6 Diagrama UML de contratos

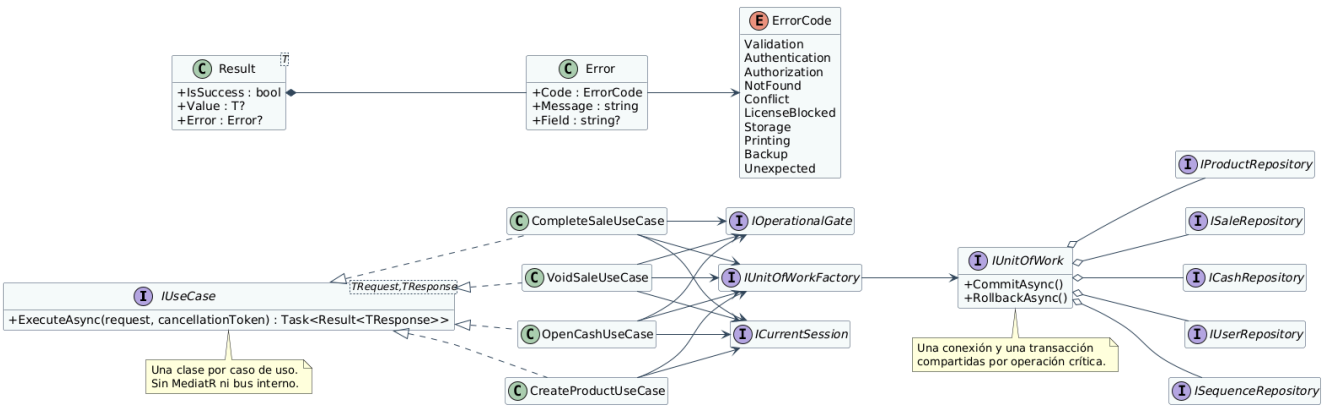


Figura 2. Casos de uso, Result<T>, sesión y unidad de trabajo.

6. Diseño del dominio

6.1 Tipos numéricos

MoneyCop encapsulará un long de pesos enteros. ScaledQuantity encapsulará un long en milésimas. Las operaciones críticas usarán checked para detectar desbordamiento. No se utilizarán double, float ni decimal para persistir o calcular importes y cantidades aprobadas.

6.2 Objetos de valor

• Plu: Trim, mayúsculas, 1 a 32 caracteres alfanuméricos.

• ProductName: Trim, 1 a 120 caracteres.

• UserDisplayName: Trim, 1 a 60 caracteres.

• MoneyCop: importe entero con la condición de signo exigida por cada regla.

• ScaledQuantity: milésimas exactas; para UNIT debe ser múltiplo de 1000.

• HistoricalTimestamp: Unix UTC en milisegundos y desplazamiento local entre -840 y 840 minutos.

• SaleNumber: entero positivo y formato visible V- con mínimo seis dígitos.

6.3 Enumeraciones

ProductUnit tendrá KG, LB y UNIT. UserRole tendrá ADMIN y CASHIER. PaymentMethod tendrá CASH y TRANSFER. Los valores persistidos se mapearán de forma explícita a los TEXT definidos en 001_Initial.sql.

6.4 Producto

Product mantendrá ProductId, Plu, Name, Unit, PriceCop, StockMilli, IsActive y UnitLocked. La unidad no cambiará cuando UnitLocked sea verdadero. UnitLocked solo cambiará de falso a verdadero. La existencia podrá ser negativa.

6.5 Venta en preparación

SaleDraft y SaleDraftLine existirán en memoria y no se persistirán como tablas. Cada línea contendrá ProductId y QuantityMilli; el precio, nombre, PLU, unidad y actividad se recargarán al confirmar para construir snapshots consistentes. La venta en preparación no reservará inventario.

6.6 Cálculo de línea

LinePricing calculará (UnitPriceCop × QuantityMilli + 500) / 1000 con enteros positivos y contexto checked. El subtotal será la suma de líneas ya redondeadas. El descuento general estará entre cero y subtotal. El total será subtotal menos descuento.

6.7 Pago

Para CASH se exigirá ReceivedCop mayor o igual al total y ChangeCop será ReceivedCop menos TotalCop. Para TRANSFER no existirán ReceivedCop ni ChangeCop. No habrá pagos combinados.

6.8 Historial

Sale, SaleLine y SaleVoid se tratarán como datos históricos inmutables. La anulación será una entidad separada y no una modificación de Sale. Los modelos de lectura derivarán ACTIVE o VOIDED de la presencia de SaleVoid.

7. Catálogo de casos de uso

7.1 Inicio y licenciamiento

• InitializeApplicationUseCase: coordinar validaciones posteriores a la composición técnica.
• GetLicenseStatusUseCase: obtener estado seguro de prueba o licencia.
• ActivateLicenseUseCase: validar y guardar una licencia vitalicia offline.
• CreateFirstAdministratorUseCase: habilitado solo cuando User está vacío.

7.2 Autenticación y sesión

• ListActiveUsersUseCase: listar únicamente usuarios activos para selección.
• AuthenticateUserUseCase: verificar PIN con comparación constante y crear la sesión en memoria.
• SignOutUseCase: limpiar la sesión y volver al selector de usuarios.

7.3 Usuarios, solo administrador

• ListUsersUseCase y GetUserUseCase.
• CreateUserUseCase.
• UpdateUserUseCase para nombre y rol.
• ResetUserPinUseCase.
• SetUserActiveStateUseCase.
• DeleteUserUseCase, permitido solo sin referencias históricas.

Todos aplicarán DisplayName único NOCASE y la protección del último administrador activo.

7.4 Productos e inventario, solo administrador

• SearchProductsUseCase y GetProductUseCase.
• CreateProductUseCase.
• UpdateProductUseCase.
• SetProductActiveStateUseCase.
• DeleteProductUseCase, solo sin SaleLine asociada.
• AdjustInventoryUseCase, reemplazo directo de StockMilli sin motivo ni historial auditable.

Las operaciones activarán UnitLocked cuando corresponda y respetarán la independencia de cada existencia.

7.5 Caja, administrador o cajero

• GetOpenCashSessionUseCase.
• OpenCashSessionUseCase.
• GetCashClosingSummaryUseCase.

• CloseCashSessionUseCase.

No se solicitará efectivo contado ni se calcularán diferencias. Los campos de cierre serán una instantánea inmutable.

7.6 Punto de venta, administrador o cajero

• SearchSellableProductsUseCase: buscar solo productos activos por PLU o nombre.

• CompleteSaleUseCase: confirmar y persistir la venta completa.

• PrintCompletedSaleReceiptUseCase: imprimir desde el resultado de una venta ya confirmada cuando el usuario lo solicite.

La edición del borrador y el cálculo preliminar podrán ejecutarse en el ViewModel mediante servicios puros de Domain, pero CompleteSaleUseCase repetirá todas las validaciones con datos actuales.

7.7 Historial y anulación, solo administrador

• SearchSaleHistoryUseCase.

• GetSaleDetailsUseCase.

• VoidSaleUseCase.

La búsqueda no constituye un módulo de reportes y solo expondrá los filtros aprobados por el futuro diseño de interfaz. VoidSaleUseCase exigirá venta vigente y caja todavía abierta.

7.8 Configuración e impresión, solo administrador

• GetBusinessSettingsUseCase.

• UpdateBusinessSettingsUseCase.

• ListInstalledPrintersUseCase.

Los campos vacíos se normalizarán a NULL. La impresora será opcional y deberá existir en Windows al intentar imprimir.

7.9 Respaldo, solo administrador

• CreateBackupUseCase: recibir destino elegido, crear temporal mediante Backup API, ejecutar integrity_check y publicar solo una copia válida.

No existirá RestoreBackupUseCase en la interfaz.

8. Persistencia y unidad de trabajo

8.1 Puertos

Application declarará interfaces específicas como IUserRepository, IProductRepository, ICashRepository, ISaleRepository, ISequenceRepository, IBusinessSettingsRepository, IUnitOfWork e IUnitOfWorkFactory. No se utilizará un repositorio genérico CRUD.

8.2 Unidad de trabajo

IUnitOfWork agrupará repositorios que comparten una única DbConnection y DbTransaction. IUnitOfWorkFactory permitirá modos ReadOnly, Write y WriteImmediate. CompleteSale, VoidSale, OpenCashSession y CloseCashSession utilizarán WriteImmediate cuando deban adquirir el bloqueo de escritura antes de leer y decidir.

8.3 Propiedad de recursos

Infrastructure será responsable de abrir, configurar y liberar conexiones y transacciones. CommitAsync marcará la unidad como completada. DisposeAsync sin commit ejecutará rollback. Los repositorios nunca crearán transacciones independientes dentro de una unidad de trabajo activa.

8.4 Conexiones SQLite

Cada conexión habilitará foreign_keys, busy_timeout de 5000 ms y los parámetros aprobados. WAL y synchronous FULL se verificarán durante inicialización. Todo SQL será constante o recurso controlado y usará parámetros para valores externos.

8.5 Mapeo

Las filas Dapper se mapearán a records internos de persistencia y luego a DTO o tipos de Domain mediante funciones explícitas. No se expondrán modelos de tabla directamente a Desktop. Los enums, booleanos, cantidades, dinero y fechas tendrán conversiones centralizadas y probadas.

8.6 Consultas

Las búsquedas de listas serán de solo lectura y proyectarán únicamente las columnas necesarias. Se aprovecharán los índices aprobados. No se cargarán 100.000 ventas completas en memoria; las consultas de historial deberán ser acotadas por los criterios que apruebe el diseño de interfaz.

9. Secuencias transaccionales críticas

9.1 Inicio de aplicación

1. Adquirir el mutex de instancia única.
2. Inicializar rutas bajo ProgramData y el registro técnico.
3. Abrir SQLite y aplicar parámetros de conexión.
4. Validar versión y checksums de migraciones.
5. Si hay migraciones pendientes, crear respaldo consistente, ejecutar cada script en transacción y registrar SchemaMigration.
6. Reconciliar las copias DPAPI de SQLite y ProgramData y evaluar fecha, prueba y licencia.
7. Si User está vacío, mostrar el asistente del primer administrador; de lo contrario, mostrar los usuarios activos.
8. Exponer a Desktop el estado operativo sin borrar datos ante un bloqueo.

9.2 Diagrama de inicio

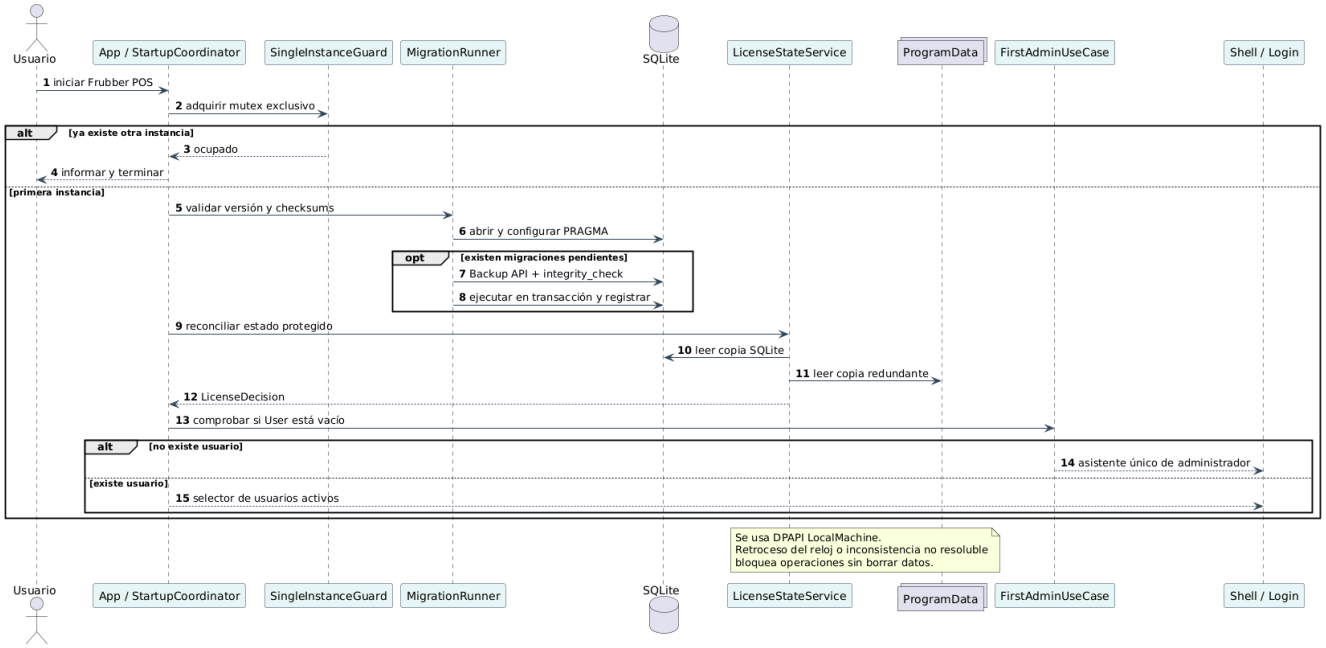


Figura 3. Instancia única, migraciones, licencia y primer administrador.

9.3 Confirmación de venta

1. Exigir sesión ADMIN o CASHIER y validar el estado operativo.
2. Abrir una unidad WriteImmediate.
3. Confirmar que existe una caja abierta y que el usuario sigue activo.
4. Recargar por ProductId todos los productos y exigir que estén activos.
5. Validar unidades y cantidades; calcular líneas, subtotal, descuento, total y pago.
6. Incrementar SystemSequence.SaleNumber dentro de la transacción.
7. Insertar Sale con snapshots del cajero y del instante.
8. Insertar SaleLine con snapshots completos y orden estable.
9. Descontar QuantityMilli de Product.StockMilli y establecer UnitLocked = 1.
10. Confirmar la transacción y devolver CompletedSaleDto.
11. Mostrar confirmación. Imprimir solo si el usuario lo solicita y siempre después del commit.

9.4 Diagrama de venta

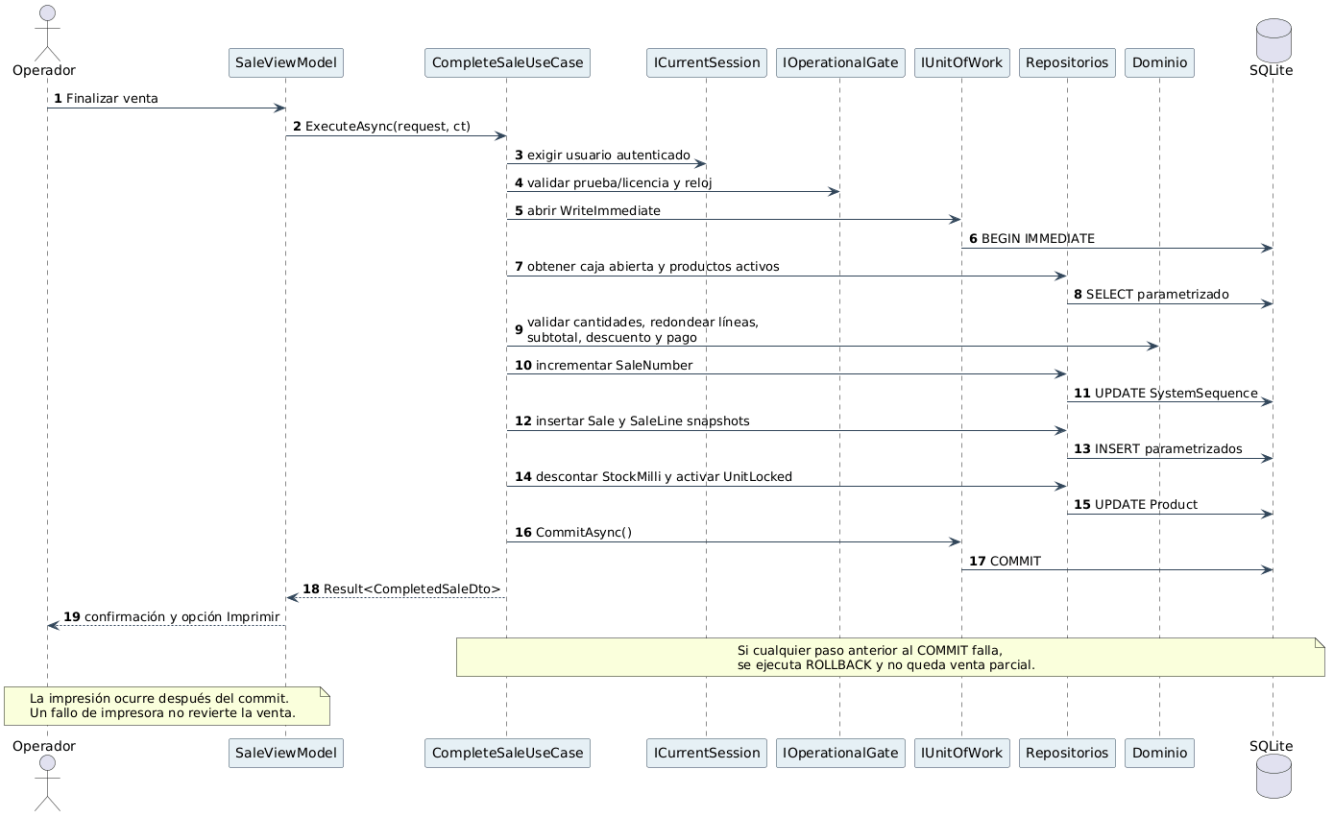


Figura 4. CompleteSaleUseCase y límites del commit.

9.5 Anulación completa

1. Exigir sesión ADMIN y validar el estado operativo.
2. Abrir una unidad WriteImmediate.
3. Cargar Sale, líneas, SaleVoid y CashSession.
4. Rechazar si la venta no existe, ya está anulada o su caja está cerrada.
5. Insertar SaleVoid con administrador y fecha/hora, sin motivo.
6. Reintegrar cada QuantityMilli a su Product.
7. Confirmar todo o revertir todo.

9.6 Diagrama de anulación

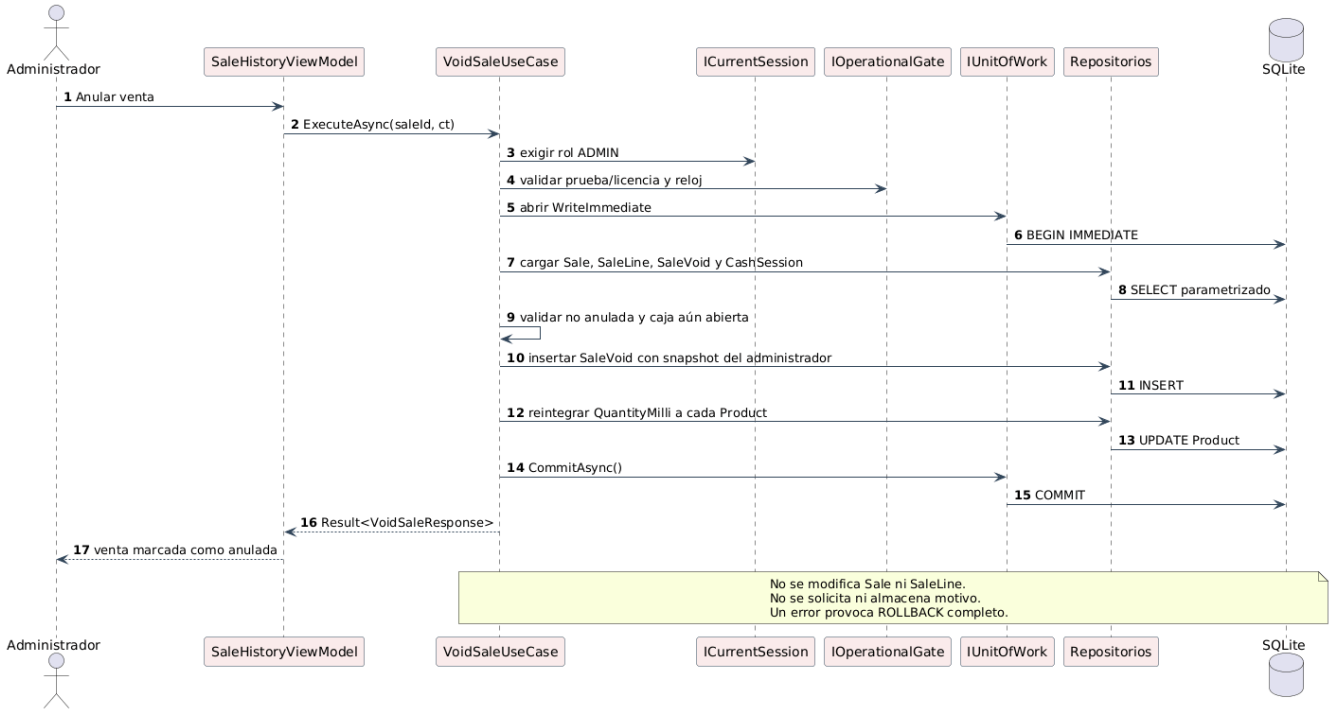


Figura 5. VoidSaleUseCase e inventario.

9.7 Apertura de caja

OpenCashSessionUseCase validará saldo inicial no negativo y ejecutará la inserción en WriteImmediate. El índice único parcial resolverá cualquier carrera residual como Conflict.

9.8 Cierre de caja

CloseCashSessionUseCase calculará dentro de WriteImmediate los totales vigentes, escribirá todos los campos Closed en un único UPDATE y confirmará. No habrá anulaciones posteriores sobre ventas de esa caja.

9.9 Actualización del último administrador

La eliminación, desactivación o cambio de rol cargará dentro de la transacción el número de administradores activos. Si el usuario objetivo es el último, devolverá Conflict sin modificar datos.

10. Diseño de Desktop y MVVM

10.1 ViewModels

Cada pantalla tendrá un ViewModel enfocado. Los ViewModels expondrán estado observable, validación visual y comandos; no abrirán conexiones, no ejecutarán SQL y no construirán hashes ni firmas.

10.2 Comandos asíncronos

AsyncCommand impedirá ejecuciones duplicadas mientras esté ocupado, expondrá IsRunning y manejará cancelación cuando sea segura. Los cambios de propiedades enlazadas se publicarán en el Dispatcher de WPF.

10.3 Navegación y diálogos

Desktop declarará adaptadores de presentación para navegación, mensajes, selección de archivo y confirmaciones. Application no referenciará MessageBox, Window, Dispatcher ni tipos de diálogo. El diseño visual y las rutas exactas se definirán en el documento de interfaz.

10.4 Sesión

La sesión autenticada vivirá únicamente en memoria mediante ICurrentSession. Contendrá UserId, DisplayName y UserRole; nunca el PIN. SignOut la eliminará. Cada caso de uso protegido comprobará el rol aunque la opción no sea visible en la interfaz.

10.5 Estado de borrador de venta

El borrador permanecerá en memoria del ViewModel. Agregar o quitar líneas no escribirá en SQLite. Si CompleteSale falla, el borrador seguirá disponible para corregirlo, salvo que el usuario decida descartarlo.

11. Seguridad

11.1 PIN

IPinHasher de Application será implementado en Infrastructure mediante PBKDF2-HMAC-SHA-256, sal aleatoria de 16 bytes, hash de 32 bytes, iteraciones calibradas y versión. AuthenticateUserUseCase usará comparación constante. No habrá PIN predeterminado.

11.2 Autorización

La autorización se aplicará en Application antes de acceder a puertos de escritura. La ocultación de botones será solo una medida de experiencia, nunca el control principal.

11.3 Licencia y reloj

IOperationalGate evaluará la decisión producida por el servicio de licencia. La validación se repetirá antes de abrir o cerrar caja, completar o anular una venta, según la arquitectura aprobada. ActivateLicenseUseCase será la única operación que aceptará una clave y nunca expondrá la clave privada.

11.4 Datos sensibles y logs

No se registrarán PIN, sal, hash, clave completa, carga DPAPI descifrada, firma privada ni identificadores físicos completos. Los errores visibles no incluirán SQL, rutas internas sensibles ni trazas.

12. Fechas y reloj

IClock devolverá DateTimeOffset actual y permitirá obtener Unix UTC en milisegundos y desplazamiento local. Los casos históricos capturarán ambos valores una sola vez por operación. Las pruebas usarán un reloj falso; Domain no llamará directamente a DateTime.Now.

13. Impresión

13.1 Contratos

IReceiptComposer convertirá CompletedSaleDto y BusinessSetting en un modelo imprimible. IReceiptPrinter enviará ese modelo a System.Drawing.Printing mediante la impresora configurada.

13.2 Orden de ejecución

La venta se confirmará antes de imprimir. PrintCompletedSaleReceiptUseCase nunca abrirá ni revertirá la transacción comercial. Un fallo devolverá Printing y permitirá informar que la venta quedó guardada.

13.3 Contenido

El comprobante utilizará los snapshots de la venta, no el catálogo actual. Mostrará únicamente los datos aprobados, y para efectivo incluirá recibido y cambio. No solicitará datos de cliente.

14. Respaldo

CreateBackupUseCase exigirá ADMIN, validará el destino y delegará en ISqliteBackupService. Infrastructure escribirá primero un archivo temporal, ejecutará integrity_check sobre la copia y solo entonces la moverá al nombre final. Un fallo eliminará el temporal cuando sea posible, preservará la base original y devolverá Backup.

15. Migraciones e inicio seguro

MigrationRunner enumerará recursos 001_, 002_ y siguientes en orden numérico, calculará SHA-256 y comparará SchemaMigration. Rechazará versiones futuras y checksums distintos. Antes del primer script pendiente creará una copia consistente. Cada script será inmutable y se ejecutará en su propia transacción controlada por el ejecutor; el script no incluirá BEGIN ni COMMIT.

16. Registro y diagnóstico

Serilog escribirá bajo ProgramData con rotación diaria, retención de 15 días y límite de tamaño según la arquitectura. Cada operación crítica tendrá un identificador de correlación técnico. Se registrarán inicio, versión, migraciones, respaldos, fallos de base, impresión, intentos de acceso fallidos y bloqueos de licencia o reloj sin datos sensibles.

17. Concurrencia y resiliencia

17.1 Instancia única

El mutex evita dos interfaces escribiendo a la vez. SQLite conservará WAL, busy_timeout y restricciones como defensa adicional. No se diseñará sincronización entre computadores.

17.2 Estado obsoleto

Los casos de uso de escritura volverán a cargar el estado requerido dentro de su transacción. Si cambió respecto a lo mostrado, devolverán Conflict y la interfaz actualizará los datos.

17.3 Fallos de almacenamiento

Un error antes de commit provoca rollback. Si el resultado del commit fuera incierto por un fallo excepcional, se cerrará la conexión, se registrará el incidente y la interfaz deberá recargar el estado antes de permitir repetir la operación.

17.4 Recuperación

La aplicación no intentará reparar silenciosamente una base cuyo integrity_check falle. Bloqueará la operación normal, preservará archivos y mostrará una referencia diagnóstica para procedimiento técnico externo.

18. Estrategia de pruebas del diseño

18.1 Domain

Pruebas de MoneyCop, ScaledQuantity, unidades, redondeo exacto de 0,50 hacia arriba, descuentos, efectivo y cambio, formato de consecutivo y desbordamientos.

18.2 Application

Pruebas de cada caso de uso con puertos falsos: permisos, Result<T>, primer administrador, último administrador, producto, ajuste, caja, venta, anulación, impresión posterior al commit, licencia y cancelación.

18.3 Infrastructure

Pruebas con SQLite temporal para 001_Initial.sql, restricciones STRICT, índices, vistas, triggers, transacciones, rollback, consultas Dapper, migraciones y Backup API. Pruebas específicas para PBKDF2, ECDSA, DPAPI cuando Windows esté disponible, reconciliación de estado y reloj.

18.4 Desktop

Pruebas de ViewModels sin ventanas reales: comandos, estados ocupados, mensajes derivados de ErrorCode, sesión, borrador de venta y navegación. La interfaz gráfica se verificará adicionalmente mediante pruebas manuales definidas en el plan de pruebas.

19. Trazabilidad resumida

• Autenticación y usuarios: RF-AUT, RF-USR, RN-USR, ADR-011.
• Productos, unidades e inventario: RF-PRO, RF-UNI, RF-INV; RN-PRO, RN-UNI, RN-INV; ADR-009.
• Caja, ventas y pagos: RF-CAJ, RF-VEN; RN-CAJ, RN-VEN; ADR-005 y ADR-010.
• Historial y anulaciones: RF-HIS, RF-ANU; RN-HIS; Diseño de base de datos 1.0.
• Impresión y configuración: RF-IMP, RN-IMP, ADR-019.
• Respaldo y migraciones: RN-BKP, ADR-008, ADR-010 y ADR-015.
• Licencia y reloj: RF-LIC, RN-LIC, ADR-012 a ADR-014.
• Estructura y dependencias: ADR-004, ADR-020 y sección 16 de Arquitectura técnica 1.0.

20. Decisiones incorporadas

P-DD-001 = A. Una clase por caso de uso, sin MediatR.
P-DD-002 = A. Microsoft.Extensions.DependencyInjection.
P-DD-003 = A. Result<T> tipado para fallos esperados.
P-DD-004 = A. Instancia única mediante mutex; segunda ejecución informa y termina.
P-DD-005 = A. async/await y CancellationToken de extremo a extremo.
P-DD-006 = A. Asistente de primer inicio para crear el primer administrador.
P-DD-007 = A. Protección del último administrador activo.
No quedan decisiones abiertas identificadas en esta versión aprobada.

21. Aprobación

Versión aprobada: 1.0
Estado de aprobación: Aprobado
Responsable de aprobación: Jhoan Rivera
Fecha de aprobación: 28 de septiembre de 2026
Fin del documento